

Documento Mestre v1.3.0

Receita tecnica, epistemologica e operacional do ARCA System 0.3.0

CORE	1.0.0
SISTEMA	0.3.0
VERIFICACAO	42 / 42 testes locais
DISTRIBUICAO	publica higienizada

Sumario

1. Finalidade e autoridade	3
2. Estado consolidado da versao 0.3.0	3
3. Fronteiras arquiteturais	3
4. Modelo canonico do Core	4
5. Invariantes epistemologicos	5
6. Event log e integridade	5
7. TRACE, invalidacao, reavaliacao e O Limite	5
8. Operacoes do Core e CLI	6
9. Workbench local	6
10. Agent Bundle e revisao humana	7
11. Acquisition Adapter e cadeia de custodia	7
12. Seguranca operacional	8
13. Conformidade ACS	8
14. Testes e reproducao	9
15. Politica de privacidade e distribuicao	9
16. Estrutura da distribuicao	9
17. Governanca e atualizacao	10
18. Roteiro recomendado	10
19. Registro da edicao 1.3.0	10
20. Declaracao final	11

1. Finalidade e autoridade

O ARCA e um metodo e uma arquitetura local para transformar uma pergunta em um grafo auditavel de fontes, documentos, informacoes, proposicoes, hipoteses, lacunas e conclusoes proporcionais. O objetivo nao e fabricar certeza. O objetivo e conservar a trilha que permite verificar de onde cada afirmacao veio, como foi tratada, quais dependencias sustentam o resultado e o que ainda poderia muda-lo.

Este documento e a referencia consolidada da edicao 1.3.0. Em caso de conflito, aplica-se a seguinte precedencia:

- 1 decisao humana expressamente ratificada e ADR vigente;
- 2 especificacao normativa mais recente e compativel;
- 3 implementacao integrada e testes reproduziveis;
- 4 relatorio de entrega e manifesto de build;
- 5 material historico, demonstrativo ou superado.

A existencia de um arquivo nao o torna normativo. Um exemplo nao cria regra. Um teste confirma apenas o comportamento que efetivamente exercita. Um hash detecta alteracao dos bytes, mas nao prova autenticidade externa nem verdade do conteudo.

2. Estado consolidado da versao 0.3.0

O marco 0.3.0 integra quatro superficies funcionais:

- ARCA Core: ontologia, IDs, relacoes, eventos, TRACE, invalidacao, reavaliacao, O Limite, ACS, importacao legada e exportacao;
- ARCA Workbench: interface local responsiva e API de uso cotidiano;
- Agent Bundle: contrato portatil de proposta unica com revisao humana;
- Acquisition Adapter: captura local autorizada, SHA-256 real e cadeia de custodia separada.

O sistema funciona offline e nao possui dependencia externa de runtime. A suite declara 42 testes: 30 da base Core, CLI, Workbench e Agent Bundle, mais 12 de aquisicao e custodia. A arvore higienizada foi executada localmente com 42 aprovacoes, nenhuma falha, nenhum teste ignorado e nenhum cancelamento.

Este resultado autoriza chamar o marco de funcional local verificado. Nao autoriza afirmar prontidao de producao, conformidade juridica, seguranca completa, autenticidade documental externa ou elegibilidade ao selo integral.

3. Fronteiras arquiteturais

O nucleo deve permanecer pequeno, deterministico e independente de rede. Tudo que conversa com navegador, provedor de IA, armazenamento remoto, OCR, assinatura, publicacao ou busca pertence a uma borda substituivel.

Fluxo de autoridade:

- 1 uma pessoa formula a questao e declara escopo e limites;
- 2 fontes e documentos sao registrados sem promocao automatica a verdade;
- 3 informacoes sao extraidas com localizador e transformacao declarados;
- 4 relacoes explicitas ligam informacoes a proposicoes;
- 5 hipoteses concorrentes e lacunas permanecem visiveis;
- 6 um agente pode preparar uma proposta, mas nao gravar o estado canonico;
- 7 uma pessoa revisa a proposta;
- 8 apenas o Core valida a operacao e acrescenta um evento;
- 9 conclusoes so podem ser fechadas depois do Crivo e de O Limite.

As camadas publicas de Publisher, Registry, Attestation, Resolver e Federation continuam fora do executavel 0.3.0. Elas podem ser integradas no futuro sem transferir autoridade epistemica para a infraestrutura.

4. Modelo canonico do Core

O estado canonico usa os seguintes tipos:

Tipo	Papel
INV	Investigacao, objetivo, escopo e limites
Q	Questao verificavel de primeira classe
SRC	Fonte ou origem declarada
DOC	Documento adquirido ou referenciado
INF	Informacao extraida de um documento
PRO	Proposicao submetida a classificacao epistemica
ENT	Entidade identificada no escopo
EVT	Evento do mundo investigado, distinto do event log
HIP	Hipotese e alternativas concorrentes
CON	Conclusao proporcional e reabrivel
FRM	Enquadramento analitico declarado
GAP	Lacuna material
SEA	Busca executada e seu resultado
REL	Relacao tipada entre objetos existentes

A questao Q nao e texto decorativo. Ela e um no ligado a investigacao e as conclusoes. Evidencia tambem nao e um tipo de objeto independente: no contrato canonico, evidencia e o efeito explicito de uma relacao INF -> PRO.

Isso impede dois atalhos perigosos:

- tratar um documento como prova direta de uma conclusao;

- tratar uma informacao extraida como fato sem registrar qual proposicao ela sustenta, enfraquece ou contradiz.

5. Invariantes epistemologicos

As invariantes abaixo orientam implementacao, revisao e auditoria:

- 1 toda investigacao nasce de uma questao verificavel;
- 2 documento nao e verdade;
- 3 informacao deve apontar para documento e localizador;
- 4 evidencia exige relacao explicita `INF -> PRO`;
- 5 classificacao epistemica exige justificativa;
- 6 ausencia de evidencia nao vira evidencia de ausencia;
- 7 desconhecido nao vira falso;
- 8 invalidacao de suporte nao cria contradicao automaticamente;
- 9 hipoteses concorrentes permanecem representadas;
- 10 lacunas materiais permanecem visiveis;
- 11 toda conclusao declara bases favoraveis, contrarias e limitacoes;
- 12 toda conclusao pode declarar condicoes de reabertura;
- 13 TRACE mostra caminhos existentes e nao inventa conexoes;
- 14 o agente propoe e a pessoa decide;
- 15 o Core e a unica autoridade de escrita canonica.

Os estados qualitativos suportados sao `Confirmado`, `Provavel`, `Possivel`, `Nao verificado`, `Contradito` e `Desconhecido`. O mapeamento de formatos legados e conservador: quando nao ha equivalente seguro, o resultado e `Desconhecido`.

6. Event log e integridade

Cada alteracao canonica e representada por um evento NDJSON append-only. O evento registra investigacao, sequencia, operacao, horario, ator, payload, hash anterior e seu proprio SHA-256. A serializacao canonica torna o calculo reproduzivel.

O estado visivel e uma projecao reconstruida a partir do log. Uma cadeia adulterada interrompe a leitura em vez de produzir silenciosamente uma versao plausivel. Escritas usam lock, arquivo temporario, sincronizacao e rename atomico. A concorrencia otimista e conferida dentro do bloqueio por `expectedEventHead`.

O hash local oferece integridade de conteudo. Ele nao substitui assinatura digital, carimbo de tempo confiavel, identidade verificada, backup, criptografia de repouso ou auditoria externa.

7. TRACE, invalidacao, reavaliacao e O Limite

TRACE percorre somente nos e arestas existentes e conserva todos os caminhos relevantes ate o alvo. O algoritmo detecta ciclos, informa truncamento e evita loops infinitos. Um grafo visual nunca pode acrescentar um elo que nao existe no estado.

Quando um documento, informacao ou proposicao e invalidado, dependentes sao marcados para reavaliacao. A remocao de suporte reduz a confianca de forma conservadora; ela nao transforma automaticamente a afirmacao em **Contradito**.

O Limite impede o fechamento de uma conclusao quando existe lacuna material com potencial de alterar o resultado. Uma execucao exige justificativa humana explicita, preservada no event log. Fechamento nao significa imutabilidade: as condicoes de reabertura continuam parte do objeto.

8. Operacoes do Core e CLI

As operacoes principais sao:

- criar e consultar investigacao;
- acrescentar objeto tipado;
- criar relacao entre objetos existentes;
- calcular TRACE;
- invalidar e reavaliar;
- validar perfis ACS;
- fechar conclusao sob O Limite;
- importar formato legado de maneira conservadora;
- exportar estado, eventos, validacao e hashes.

Exemplo local:

```
npm ci
npm run arca -- investigation create --home .arca \
  --question "Qual afirmacao deve ser verificada?" \
  --objective "Verificar com material rastreado" \
  --scope "Fontes legitimamente acessiveis" \
  --limits "Sem conteudo privado"
npm run arca -- validate --home .arca --investigation INV-000001
```

A importacao legado permanece suportada pela API, mas esta distribuicao nao inclui estados ou exportacoes de investigacoes de desenvolvimento. Os testes constroem fixtures ficticias apenas na memoria e as apagam ao terminar.

9. Workbench local

O Workbench fornece formularios para os tipos canonicos, mapa SVG, inventario, TRACE, conformidade, historico, exportacao e revisao de propostas. Ele e cliente do Core, nao uma segunda implementacao de regras epistemicas.

Por padrao, o servidor escuta somente 127.0.0.1. A API exige token CSRF, verifica **Origin**, limita o corpo a 1 MiB, usa CSP, bloqueia frames e aplica uma lista fechada de arquivos estaticos. O modo remoto e recusado sem opcao explicita. Mesmo quando habilitado, nao ha autenticao embutida; por

isso deve ficar atras de uma camada confiavel e nao e configuracao de producao.

```
npm run workbench
```

A interface fica disponivel em <http://127.0.0.1:4317> e usa `.arca-workbench` como armazenamento padrao.

10. Agent Bundle e revisao humana

O Agent Bundle define um prompt portatil e o schema `arca-agent-proposal-v1`. Uma proposta contem exatamente uma operacao, premissas, incertezas, cabeca esperada do log e a declaracao `humanReviewRequired: true`.

O ciclo seguro e:

- 1 agente observa o estado recebido;
- 2 agente prepara uma unica operacao;
- 3 Workbench calcula e conserva hashes da proposta;
- 4 pessoa revisa conteudo, premissas e incertezas;
- 5 confirmacao textual autoriza a aplicacao;
- 6 Core confere novamente `expectedEventHead` dentro do lock;
- 7 evento aplicado registra revisor, ID e hash da proposta.

Se o grafo mudar, a proposta fica obsoleta e falha fechada. Rejeicao e persistida e impede aplicacao posterior. O agente nao recebe autoridade para adquirir, assinar, publicar, conceder selo ou reescrever o log.

11. Acquisition Adapter e cadeia de custodia

O adaptador 0.3.0 captura apenas arquivo regular dentro de uma raiz autorizada. Ele preserva os bytes, calcula SHA-256 sobre o conteudo realmente lido, registra proveniencia declarada e devolve uma proposta de `DOC`. Nenhuma requisicao de rede e executada.

O fluxo de captura:

- 1 validar IDs e campos fechados;
- 2 exigir base e declaracao de acesso;
- 3 resolver fisicamente a raiz autorizada;
- 4 rejeitar traversal e link simbolico;
- 5 abrir sem seguir link quando a plataforma permite;
- 6 limitar tamanho antes e durante a leitura;
- 7 detectar mudanca concorrente;
- 8 copiar bytes por escrita atomica;
- 9 calcular SHA-256;
- 10 registrar `ACQUISITION_CAPTURED`;

- 11 projetar `manifest.json`;
- 12 produzir proposta sujeita a revisao humana.

Transformacoes criam derivados e registram hash de entrada, hash de saida, ferramenta, versao, parametros, horario e ator. Revisoes acrescentam `REVIEW_RECORDED`. O `custody.ndjson` e a fonte de verdade; o manifesto e uma projecao verificavel.

Captura prova quais bytes foram preservados e como foram tratados. Nao prova que o conteudo e autentico, completo, verdadeiro ou suficiente.

12. Seguranca operacional

Controles presentes:

- validacao de IDs antes de formar caminhos;
- defesa contra traversal e escapes por symlink;
- permissoes privadas quando suportadas;
- lock de escrita e concorrencia otimista;
- escrita atomica de eventos, exportacoes e propostas;
- hash chain para eventos do Core e da custodia;
- CSP, CSRF, politica de origem e limite de corpo no Workbench;
- proposta de uma operacao com revisao humana obrigatoria;
- falha fechada diante de adulteracao ou proposta obsoleta;
- zero rede no Core e no Acquisition Adapter.

Lacunas antes de producao:

- threat model formal e fuzzing amplo;
- autenticacao e autorizacao multiusuario;
- criptografia de volume e gestao de chaves;
- assinatura digital e timestamp externo confiavel;
- backup, restauracao e resposta a incidentes testados;
- auditoria semantica e de seguranca independente;
- politica publica de licenca, marca e governanca.

13. Conformidade ACS

O validador emite 47 requisitos e perfis `ACS-S`, `ACS-E`, `ACS-O`, `ACS-P` e `ACS-FULL`. Estados nao verificaveis automaticamente permanecem `UNSPECIFIED` ou `PARTIAL`; o sistema nao os converte em `PASS` por conveniencia.

Um resultado estrutural correto pode e deve recusar o selo integral quando faltam verificacoes semanticas, auditoria independente ou controles operacionais. A ausencia de selo e um resultado informativo, nao uma falha do motor.

14. Testes e reproducao

A suite 0.3.0 cobre:

- 12 testes de Core e migracao sintetica;
- 3 testes de CLI;
- 7 testes de propostas e revisao de agente;
- 7 testes de Workbench e API;
- 1 teste de propriedade com 40 DAGs pseudoaleatorios;
- 12 testes numerados 31 a 42 para aquisicao e custodia.

```
npm ci
npm test
npm run check
npm run test:acquisition
```

O workflow `.github/workflows/arca-ci.yml` executa `npm ci`, `npm test` e `npm run check` em Node.js 22.18.0. O portao exige exatamente 42 testes e valida arquivos obrigatorios, nove ADRs, seis arquivos de teste e a cadeia da captura publica.

15. Politica de privacidade e distribuicao

A arvore publica segue minimizacao de dados:

- nao inclui nomes de pessoas relacionados a casos;
- nao inclui estados, relatorios ou exportacoes de investigacoes;
- nao inclui localizadores privados de arquivos;
- nao inclui tokens, credenciais ou diretorios de trabalho;
- fixtures de teste sao ficticias, efemerias e marcadas como simulacao;
- a captura distribuida usa apenas a edicao publica deste documento;
- o revisor e identificado pelo papel, nao por nome civil.

Uma investigacao real deve permanecer fora do repositorio de software. Quando for necessario compartilhar um resultado, o Publisher futuro devera separar conteudo selado de projecao publica, aplicar redacao verificavel e produzir novo manifesto. Apagar apenas o nome do arquivo nao e higienizacao suficiente.

16. Estrutura da distribuicao

<code>.github/workflows/</code>	portao reproduzivel de CI
<code>docs/</code>	especificacoes, ADRs e relatorios
<code>schemas/</code>	contratos JSON canonicos
<code>packages/core/</code>	Core deterministico e event store
<code>packages/cli/</code>	interface offline
<code>packages/workbench/</code>	servidor local, API e interface visual
<code>packages/agent/</code>	protocolo e ciclo de propostas
<code>packages/acquisition/</code>	aquisicao local e custodia
<code>examples/</code>	somente fixtures tecnicas e captura publica
<code>tests/</code>	testes unitarios, integracao, propriedade e seguranca

Arquivos de investigacao, saidas locais, caches, segredos e dados pessoais nao pertencem a esta arvore.

17. Governanca e atualizacao

Mudancas de comportamento exigem:

- 1 proposta e classificacao da mudanca;
- 2 ADR quando houver decisao arquitetural ou normativa;
- 3 alteracao de implementacao e schema quando aplicavel;
- 4 teste correspondente;
- 5 atualizacao deste Documento Mestre e do changelog;
- 6 nova rodada integral em arvore limpa;
- 7 manifesto e ZIP gerados somente depois do portao verde.

SemVer documental acompanha mudancas deste corpo. SemVer do software acompanha contratos e comportamento dos pacotes. Uma revisao de privacidade que nao muda o protocolo pode manter 1.3.0 e receber a qualificacao "edicao publica higienizada".

18. Roteiro recomendado

Ordem de evolucao recomendada:

- 1 consolidar instalacao local simples para Android, Termux, desktop e Replit;
- 2 adicionar importacao de arquivos pelo Workbench sem expor rede;
- 3 definir politica de licenca, marca e contribuicao;
- 4 produzir threat model e ampliar fuzzing;
- 5 implementar cofre local, backup e restauracao;
- 6 criar Publisher com redacao e separacao publico/selado;
- 7 integrar assinatura, Registry, Attestation e Resolver como adaptadores;
- 8 executar auditorias independentes antes de qualquer alegacao de producao.

19. Registro da edicao 1.3.0

Esta edicao incorpora o Acquisition Adapter e a cadeia de custodia ao documento consolidado, registra a suite de 42 testes e estabelece uma politica explicita de distribuicao higienizada. O conteudo historico orientado a investigacoes foi retirado da superficie publica e substituido por descricoes de capacidade e fixtures sinteticas efemerias.

Arquivos normativos centrais:

- docs/ARCA_CORE_SPEC_v1.0.0.md;
- docs/ARCA_WORKBENCH_SPEC_v0.2.0.md;
- docs/ARCA_AGENT_PROTOCOL_v0.2.0.md;

- docs/ARCA_ACQUISITION_CUSTODY_SPEC_v0.3.0.md;
- schemas/arca-state-v1.schema.json;
- schemas/arca-event-v1.schema.json;
- schemas/arca-export-v1.schema.json;
- schemas/arca-acquisition-request-v1.schema.json;
- schemas/arca-custody-manifest-v1.schema.json.

ADRs ratificados:

- ADR-ARCA-0001: Core local, CLI first;
- ADR-ARCA-0002: questao como objeto de primeira classe;
- ADR-ARCA-0003: evidencia como relacao;
- ADR-ARCA-0004: event log como fonte de verdade;
- ADR-ARCA-0005: estados epistemicos qualitativos;
- ADR-ARCA-0006: fronteira Workbench/Core;
- ADR-ARCA-0007: proposta de agente e revisao humana;
- ADR-ARCA-0008: fronteira Acquisition/Core;
- ADR-ARCA-0009: bytes originais e eventos de custodia.

20. Declaracao final

O ARCA 0.3.0 e uma fundacao funcional, local e auditavel. Seu valor esta menos em produzir respostas automaticas e mais em impedir que respostas parecam mais fortes do que as evidencias permitem. O proximo ganho de confianca deve vir de melhor empacotamento, protecao operacional, separacao publico/selado e auditoria independente, nao de aumentar a autoridade de um agente.

Esta edicao publica nao contem investigacoes de desenvolvimento. O pacote pode ser reconstruido a partir do codigo, das especificacoes, dos schemas, dos testes e dos manifestos aqui identificados.